



**MANİSA**  
**CELAL BAYAR**  
**ÜNİVERSİTESİ**  
T.C.

MANİSA CELAL BAYAR ÜNİVERSİTESİ  
Mühendislik Fakültesi

Bilgisayar Mühendisliği

**Project Report**  
**JOB & CAREER RECOMMENDATION**  
**ONTOLOGY**

**Naciye Nur Akkuş – 220315039**  
**Elif Sude Yılmaz - 220315022**

**Repository:** <https://github.com/naciyenurrr/-job-ontology>

Haziran  
2026

## Table of Contents

---

|                                                                           |    |
|---------------------------------------------------------------------------|----|
| Table of Contents .....                                                   | 2  |
| 1. Research Integration.....                                              | 3  |
| 1.1 Selected Study: Ontology Population Using Large Language Models ..... | 3  |
| 1.1.1 Why This Study Was Selected.....                                    | 3  |
| 1.1.2 Relevance to Our Project .....                                      | 3  |
| 1.1.3 Integration Plan.....                                               | 4  |
| 2. Mandatory Paper Review .....                                           | 4  |
| 2.1 Core Methodology Summary .....                                        | 4  |
| 2.2 Key Contributions .....                                               | 4  |
| 2.3 Adaptation to Our Project.....                                        | 5  |
| 3. Data Acquisition.....                                                  | 5  |
| 3.1 Data Sources .....                                                    | 5  |
| 3.2 Data Collection Methodology .....                                     | 6  |
| 3.3 Preprocessing Steps .....                                             | 6  |
| 3.4 Mapping of Data to Ontology Concepts.....                             | 6  |
| 4. Ontology Usage and Expansion .....                                     | 7  |
| 4.1 Existing Ontologies Used .....                                        | 7  |
| 4.2 Methodology: METHONTOLOGY .....                                       | 7  |
| 4.3 What Was Extended in Version 2.....                                   | 8  |
| New Classes .....                                                         | 8  |
| New Object Properties .....                                               | 8  |
| New Datatype Properties .....                                             | 8  |
| New Instances .....                                                       | 9  |
| V1 → V2 Change Summary .....                                              | 9  |
| 5. Ontology Requirements Specification Document — Version 2 .....         | 9  |
| 5.1 Purpose .....                                                         | 9  |
| 5.2 Scope (Extended in V2).....                                           | 10 |
| 5.3 Implementation Language .....                                         | 10 |
| 5.4 All Competency Questions (CQ1–CQ16).....                              | 10 |
| 5.5 Non-Functional Requirements.....                                      | 10 |

|                                                                |    |
|----------------------------------------------------------------|----|
| 6. SPARQL Queries — New Competency Questions (V2).....         | 11 |
| CQ13 — Skill Proficiency Level of a Candidate .....            | 11 |
| CQ14 — Certifications Held by a Candidate .....                | 11 |
| CQ15 — Jobs Requiring a Specific Certification .....           | 12 |
| CQ16 — Candidates Meeting All Certification Requirements ..... | 12 |
| CQ11 — Skill Gap Analysis (V1, retained for reference) .....   | 12 |
| 7. GitHub Repository .....                                     | 12 |
| 8. Ontology Documentation (WIDOCO).....                        | 13 |
| 9. Submission Checklist .....                                  | 13 |

## 1. Research Integration

### 1.1 Selected Study: Ontology Population Using Large Language Models

From the materials provided under Week 12, we selected the research direction on Ontology Population Using LLMs. This approach addresses the challenge of automatically extracting and inserting concept instances into an ontology from unstructured text, using a large language model (LLM) as the extraction engine.

#### 1.1.1 Why This Study Was Selected

Our ontology's biggest limitation in Version 1 was the manual population of instances. Scaling the system to real job-market data — thousands of job postings from LinkedIn or Indeed — would require automated extraction of skills, experience requirements, and job titles from free-form text. LLM-based ontology population directly solves this bottleneck.

#### 1.1.2 Relevance to Our Project

Job postings are unstructured text documents. An LLM (e.g., GPT-4, Claude) can be prompted to parse a raw posting and return structured JSON conforming to our ontology schema. For example, given a posting for a 'Senior Data Engineer at FinAI', the LLM extracts the following structured information:

```
jobTitle      → 'Data Engineer'
requiresSkill  → [:Python, :SQL, :Spark, :CloudComputing]
requiresExperienceLevel → :Senior
offeredBy     → :FinAI
```

This JSON output is then converted to Turtle triples and appended to the ontology via a Python pipeline using rdflib. The same pipeline is applied to candidate CV parsing, enabling automated end-to-end population.

### 1.1.3 Integration Plan

| Component       | Integration Detail                                                                |
|-----------------|-----------------------------------------------------------------------------------|
| Data Source     | Raw job postings scraped from LinkedIn / Indeed APIs                              |
| LLM Prompt      | Structured extraction prompt with ontology schema (JSON Schema) as context window |
| Output Format   | JSON-LD → Turtle conversion via rdflib (Python)                                   |
| Validation      | SHACL shapes validate extracted triples before insertion into the ontology        |
| Reasoning Layer | HermiT reasoner infers :recommendedFor triples after each population batch        |

## 2. Mandatory Paper Review

### Personalized Ontology and Deep Training Tree-Based Optimal GRU-RNN for Prediction of Students' Behaviour

#### 2.1 Core Methodology Summary

The paper proposes a hybrid system combining a domain-specific personalised ontology with a Gated Recurrent Unit Recurrent Neural Network (GRU-RNN) to predict student academic behaviour. The ontology formally models student attributes, course structures, assessment patterns, and learning styles. These ontological features are fed as structured inputs to a GRU-RNN, which has been further optimised using a metaheuristic Deep Training Tree algorithm to tune hyperparameters automatically.

The system predicts three behavioural outcomes: academic performance (pass/fail), dropout risk, and engagement level. Evaluation on a university dataset showed the model outperforming baseline LSTM and standard RNN architectures on accuracy, F1-score, and RMSE metrics.

#### 2.2 Key Contributions

- A personalised ontology that captures fine-grained student attributes as knowledge graph nodes, enabling interpretable feature engineering.
- Integration of symbolic (ontology) and sub-symbolic (GRU-RNN) AI — a neurosymbolic architecture that combines the explainability of knowledge graphs with the predictive power of deep learning.

- A Deep Training Tree optimiser that avoids manual hyperparameter search for the neural component, improving reproducibility.
- Empirical demonstration that ontology-structured inputs improve neural prediction accuracy compared to flat feature vectors, validating the neurosymbolic hypothesis.

## 2.3 Adaptation to Our Project

The core idea — using an ontology as a structured feature source for a neural prediction model — maps directly onto our recruitment recommendation system:

| Paper Concept                              | Adaptation to JobOntology                                           |
|--------------------------------------------|---------------------------------------------------------------------|
| Student ontology → behaviour prediction    | Candidate ontology → job-fit score prediction                       |
| GRU-RNN trained on student history         | GRU-RNN or feedforward network trained on past application outcomes |
| Ontology features (learning style, grades) | Ontology features (skills, proficiency levels, years of experience) |
| Dropout / performance prediction           | Hiring success probability / interview invitation prediction        |
| Personalised curriculum recommendation     | Personalised job and skill-gap recommendation                       |

In practice, our ontology's structured candidate and job profiles can replace flat feature vectors as inputs to a recommendation neural network, yielding interpretable, reasoning-backed predictions — the same neurosymbolic advantage demonstrated in the paper. This represents a direct pathway from Phase 2 towards a fully AI-powered recommendation engine in future phases.

## 3. Data Acquisition

### 3.1 Data Sources

| Source                     | Type          | Description                                                                               |
|----------------------------|---------------|-------------------------------------------------------------------------------------------|
| LinkedIn Job Postings API  | API           | Real-time structured job data: title, required skills, location, seniority level.         |
| Kaggle: Job Skills Dataset | Dataset (CSV) | 150,000+ job–skill pairs scraped from Indeed; used to populate :Skill instances at scale. |

|                                 |                  |                                                                                                     |
|---------------------------------|------------------|-----------------------------------------------------------------------------------------------------|
| ESCO (European Skills Ontology) | Linked Data      | EU standard skill taxonomy reused for :TechnicalSkill and :SoftSkill alignment via skos:exactMatch. |
| LLM Extraction (GPT-4 / Claude) | Automated NLP    | Extracts structured triples from raw CV PDFs and free-form job description text.                    |
| Manual Expert Curation          | Human annotation | Domain experts verify and correct LLM-extracted triples for quality assurance.                      |

### 3.2 Data Collection Methodology

- LinkedIn API queries filtered by location (Turkey), category (Data Science, Engineering) and seniority level.
- Kaggle dataset downloaded in CSV format; columns mapped to ontology properties using a Python mapping script.
- ESCO skill URIs fetched via SPARQL from the official ESCO endpoint and aligned to our :Skill class using skos:exactMatch.
- Raw CV text parsed by a structured LLM prompt, returning JSON with candidate name, skills, years of experience, and certifications.
- All sources merged and deduplicated using a Python pipeline (pandas + rdflib) before conversion to Turtle format.

### 3.3 Preprocessing Steps

- Skill normalisation: raw string variants (e.g., 'python3', 'Python 3.x', 'py') mapped to canonical :Python URI via a normalisation dictionary.
- Experience level inference: raw integer 'years of experience' mapped to :Junior / :Mid / :Senior thresholds (0–2 / 3–5 / 6+) via a SWRL rule.
- Deduplication: identical candidate records (same name + email) merged; skill sets unioned to avoid redundant assertions.
- SHACL validation: all generated triples validated against shape constraints before loading into the main ontology graph.
- Encoding: UTF-8 enforced for all Turtle files; non-ASCII characters (Turkish names, diacritics) preserved.

### 3.4 Mapping of Data to Ontology Concepts

| Source Field      | Source       | Ontology Property        | Target Class / Range |
|-------------------|--------------|--------------------------|----------------------|
| job_title         | LinkedIn API | :jobTitle                | xsd:string           |
| required_skills[] | LinkedIn API | :requiresSkill           | :Skill               |
| seniority_level   | LinkedIn API | :requiresExperienceLevel | :ExperienceLevel     |

|                    |              |                                          |                              |
|--------------------|--------------|------------------------------------------|------------------------------|
| company_name       | LinkedIn API | :companyName                             | :Company                     |
| skill_name         | Kaggle CSV   | :Skill (IRI)                             | owl:NamedIndividual          |
| skill_category     | Kaggle CSV   | rdf:type                                 | :TechnicalSkill / :SoftSkill |
| concept_uri        | ESCO         | skos:exactMatch                          | :Skill                       |
| candidate_skills[] | LLM / CV     | :hasSkill                                | :Skill                       |
| certification_name | LLM / CV     | :holdsCertification → :certificationName | :Certification               |
| years_experience   | LLM / CV     | :yearsOfExperience                       | xsd:int                      |

## 4. Ontology Usage and Expansion

### 4.1 Existing Ontologies Used

| Ontology                  | Usage in JobOntology                                                                                                                                                    |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ESCO                      | Skill taxonomy alignment; ESCO concept URIs linked via skos:exactMatch to :Skill individuals. Source: <a href="https://esco.ec.europa.eu">https://esco.ec.europa.eu</a> |
| schema.org                | Alignment of :Job to schema:JobPosting, :Person to schema:Person, :Company to schema:Organization for broader web interoperability.                                     |
| FOAF (Friend of a Friend) | Candidate and Recruiter profiles extended with foaf:name, foaf:mbox for social graph compatibility.                                                                     |
| Dublin Core (dc:)         | Ontology metadata: dc:title, dc:creator, dc:date, dc:version — enabling catalogue-level discovery.                                                                      |
| SKOS                      | skos:definition used on all named individuals; skos:exactMatch for ESCO alignment.                                                                                      |

### 4.2 Methodology: METHONTOLOGY

Version 2 development followed the METHONTOLOGY lifecycle, a structured ontology engineering process consisting of six phases:

| Phase            | Activity in Version 2                                                                                |
|------------------|------------------------------------------------------------------------------------------------------|
| 1. Specification | ORSD v2 produced; new competency questions CQ13–CQ16 added for certifications and skill proficiency. |

|                      |                                                                                                                |
|----------------------|----------------------------------------------------------------------------------------------------------------|
| 2. Conceptualisation | Concept maps extended with :SkillProficiency, :Certification, :SkillAssessment, and :Sector classes.           |
| 3. Formalisation     | New OWL 2 DL axioms: subclass hierarchy for :Skill, disjointness constraints, domain/range declarations.       |
| 4. Implementation    | Turtle serialisation updated with V2 classes, properties, and instances. SHACL shapes authored for validation. |
| 5. Evaluation        | HermiT 1.4.3 reasoner run; all 16 CQs answered via SPARQL; SHACL validation passed with zero violations.       |
| 6. Maintenance       | Version controlled on GitHub; CHANGELOG.md documents all V1 → V2 differences with rationale.                   |

### 4.3 What Was Extended in Version 2

#### New Classes

| Class             | Description                                                                               |
|-------------------|-------------------------------------------------------------------------------------------|
| :SkillProficiency | Degree of mastery over a skill: Beginner / Intermediate / Expert.                         |
| :SkillAssessment  | Reification node linking a candidate to a skill together with a proficiency level.        |
| :Certification    | Formal credential attesting mastery of a skill or domain (e.g., Google ML Engineer cert). |
| :Sector           | Industry vertical to which a company or job belongs (e.g., FinTech, HealthTech).          |

#### New Object Properties

- :hasSkillAssessment (Candidate → SkillAssessment)
- :assessedSkill (SkillAssessment → Skill)
- :hasProficiency (SkillAssessment → SkillProficiency)
- :holdsCertification (Candidate → Certification)
- :certifiesSkill (Certification → Skill)
- :requiresCertification (Job → Certification)
- :operatesInSector (Company → Sector)
- :targetsSector (Job → Sector)

#### New Datatype Properties

- :certificationName, :issuingBody, :issueDate, :expiryDate — on :Certification
- :salary, :remote — on :Job

New Instances

- 5 new job individuals: DataEngineerJob, NLPResearcherJob + 3 extended existing jobs
- 2 new candidates: Cem Kaya, Selin Demir (LLM-populated from simulated CV text)
- 3 certification individuals: GoogleMLCert, AWSDACert, TableauDesktopCert
- 4 sector individuals: TechSector, FinTech, HealthTech, EduTech
- schema.org alignment annotations added to :Job, :Person, :Company

V1 → V2 Change Summary

| Element                  | Version 1                | Version 2                        |
|--------------------------|--------------------------|----------------------------------|
| Classes                  | 10 classes               | 14 classes (+4 new)              |
| Object Properties        | 10 properties            | 18 properties (+8 new)           |
| Datatype Properties      | 7 properties             | 13 properties (+6 new)           |
| Competency Questions     | CQ1–CQ12                 | CQ1–CQ16 (+4 new)                |
| Individuals (Jobs)       | 3 jobs                   | 5 jobs (+2 new)                  |
| Individuals (Candidates) | 3 candidates             | 5 candidates (+2 new)            |
| External Alignments      | None                     | ESCO, schema.org, FOAF, DC, SKOS |
| Validation               | HermiT consistency check | HermiT + SHACL shapes            |

5. Ontology Requirements Specification Document — Version 2

|         |                                                                                                                                                                                                                                                           |
|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Authors | Naciye Nur Akkuş (220315039)   Elif Sude Yılmaz (220315022)                                                                                                                                                                                               |
| Version | 2.0 — supersedes v1.0 (April 2026)                                                                                                                                                                                                                        |
| Date    | 12 May 2026                                                                                                                                                                                                                                               |
| Changes | Added: SkillProficiency, Certification, SkillAssessment, Sector classes; 8 new object properties; 6 new datatype properties; salary/remote datatype props; schema.org / ESCO / FOAF alignment; CQ13–CQ16; 2 new jobs, 2 new candidates, 3 certifications. |

5.1 Purpose

Provide a formal, machine-readable representation of the job recruitment domain enabling skill-based matching, personalised recommendation, and skill-gap analysis between candidates and job postings.

## 5.2 Scope (Extended in V2)

V2 extends scope to include: skill proficiency levels, formal certifications, remote work flags, salary ranges, sector classification, and alignment with ESCO / schema.org.

Out of scope: interview scheduling, salary negotiation, psychological assessment, legal compliance.

## 5.3 Implementation Language

OWL 2 DL, Turtle serialisation. Validated by HerMiT 1.4.3. Namespace:  
<http://www.example.org/jobontology#>

## 5.4 All Competency Questions (CQ1–CQ16)

| ID   | Competency Question                                                           |
|------|-------------------------------------------------------------------------------|
| CQ1  | What skills does a given candidate possess?                                   |
| CQ2  | What skills are required for a given job position?                            |
| CQ3  | Which candidates have all the skills required by a given job?                 |
| CQ4  | Which jobs match the skills of a given candidate?                             |
| CQ5  | What is the experience level of a given candidate?                            |
| CQ6  | What experience level does a given job require?                               |
| CQ7  | Which company offers a given job?                                             |
| CQ8  | Which jobs does a given company offer?                                        |
| CQ9  | Has a given candidate applied for a given job?                                |
| CQ10 | Which candidates are recommended for a given job?                             |
| CQ11 | What skills is a candidate missing for a target job (skill gap)?              |
| CQ12 | How many years of experience does a candidate have?                           |
| CQ13 | What is the proficiency level of a candidate for a given skill? ★ New in V2   |
| CQ14 | Which certifications does a candidate hold? ★ New in V2                       |
| CQ15 | Which jobs require a specific certification? ★ New in V2                      |
| CQ16 | Which candidates hold all certifications required by a given job? ★ New in V2 |

## 5.5 Non-Functional Requirements

| ID | Requirement |
|----|-------------|
|----|-------------|

|      |                                                                                                                |
|------|----------------------------------------------------------------------------------------------------------------|
| NFR1 | Decidability: OWL 2 DL profile ensures automated reasoning terminates.                                         |
| NFR2 | Consistency: no logical contradictions; verified by HermiT 1.4.3 at each version.                              |
| NFR3 | Extensibility: modular class hierarchy allows adding skills, jobs, and candidates without schema changes.      |
| NFR4 | Reusability: standard namespace (OWL 2, RDF, RDFS) ensures tool compatibility.                                 |
| NFR5 | Interoperability: Turtle serialisation is supported by all major RDF tools and reasoners.                      |
| NFR6 | Schema.org Alignment (New): Key classes aligned with schema.org for web indexability. ★ V2                     |
| NFR7 | SHACL Validation (New): All instances must pass SHACL shape constraints before publication. ★ V2               |
| NFR8 | LLM Population Support (New): Ontology structure must be expressible as a JSON schema for LLM extraction. ★ V2 |

## 6. SPARQL Queries — New Competency Questions (V2)

The following SPARQL queries demonstrate the expressiveness gained by the V2 ontology extensions. All queries use the base namespace: PREFIX : `<http://www.example.org/jobontology#>`

### CQ13 — Skill Proficiency Level of a Candidate

What is the proficiency level of Naciye for each skill she has been assessed on?

PREFIX : `<http://www.example.org/jobontology#>`

```
SELECT ?skill ?proficiency WHERE {
  :Naciye :hasSkillAssessment ?assessment .
  ?assessment :assessedSkill ?skill .
  ?assessment :hasProficiency ?proficiency .
}
```

### CQ14 — Certifications Held by a Candidate

Which certifications does Zeynep hold, and which skills do they certify?

PREFIX : `<http://www.example.org/jobontology#>`

```
SELECT ?cert ?certName ?skill WHERE {
  :Zeynep :holdsCertification ?cert .
  ?cert :certificationName ?certName .
}
```

```
?cert :certifiesSkill ?skill .  
}
```

### CQ15 — Jobs Requiring a Specific Certification

Which job postings require the Google ML Engineer certification?

```
PREFIX : <http://www.example.org/jobontology#>
```

```
SELECT ?job ?title WHERE {  
  ?job :requiresCertification :GoogleMLCert .  
  ?job :jobTitle ?title .  
}
```

### CQ16 — Candidates Meeting All Certification Requirements

Which candidates hold all certifications required by the MLEngineerJob?

```
PREFIX : <http://www.example.org/jobontology#>
```

```
SELECT ?candidate ?name WHERE {  
  ?candidate a :Candidate .  
  ?candidate :name ?name .  
  FILTER NOT EXISTS {  
    :MLEngineerJob :requiresCertification ?cert .  
    FILTER NOT EXISTS { ?candidate :holdsCertification ?cert }  
  }  
}
```

### CQ11 — Skill Gap Analysis (V1, retained for reference)

What skills is Naciye missing for the DataScientistJob?

```
PREFIX : <http://www.example.org/jobontology#>
```

```
SELECT ?missingSkill WHERE {  
  :DataScientistJob :requiresSkill ?missingSkill .  
  FILTER NOT EXISTS { :Naciye :hasSkill ?missingSkill }  
}
```

## 7. GitHub Repository

---

Repository: <https://github.com/naciyenurrr/job-ontology>

GitHub Pages (Documentation): <https://naciyenurrr.github.io/job-ontology/>

| File / Folder       | Contents                                                                                        |
|---------------------|-------------------------------------------------------------------------------------------------|
| job-ontology-v2.ttl | Main ontology file, OWL 2 DL, Version 2.0 — includes all V2 classes, properties, and instances. |
| job-ontology-v1.ttl | Archived Version 1.0 for version comparison and rollback reference.                             |
| ORSD_v2.docx        | This specification document (v2) submitted as Phase 2 deliverable.                              |
| index.html          | WIDOCO-style HTML documentation deployed via GitHub Pages.                                      |
| README.md           | Project overview, SPARQL examples, setup guide — updated for V2.                                |
| sparql/             | SPARQL query files for CQ1–CQ16 as individual .sparql files.                                    |
| shacl/              | SHACL shape constraint files for V2 instance validation.                                        |
| data/               | Raw CSV data and LLM extraction pipeline scripts (Python, rdflib).                              |
| CHANGELOG.md        | Detailed version history documenting all V1 → V2 additions and design decisions.                |

## 8. Ontology Documentation (WIDOCO)

HTML documentation was generated using WIDOCO v1.4.25. The generation command used was:

```
java -jar widoco.jar -ontFile job-ontology-v2.ttl \
  -outFolder docs/ -rewriteAll -getOntologyMetadata \
  -lang en -webVowl -noPlaceholders
```

The generated documentation is published via GitHub Pages at:

**<https://nacyenurrr.github.io/job-ontology/>**

The documentation covers all V2 classes, object properties, datatype properties, individuals, and namespace declarations. WebVOWL visualisation is included for interactive exploration of the ontology graph.

## 9. Submission Checklist

The following items are included in this Phase 2 submission:

| Requirement                                                                         | Status      |
|-------------------------------------------------------------------------------------|-------------|
| Updated report with Data Acquisition section completed (Section 3)                  | ✓ Completed |
| Selected research from Week 11/12 with relevance explanation (Section 1)            | ✓ Completed |
| Summary of mandatory paper (GRU-RNN student behaviour) (Section 2)                  | ✓ Completed |
| Updated ontology on GitHub (job-ontology-v2.ttl)                                    | ✓ Completed |
| Ontology documentation via WIDOCO (GitHub Pages)                                    | ✓ Completed |
| Specification Document Version 2 (Section 5)                                        | ✓ Completed |
| Existing ontology usage documented (ESCO, schema.org, FOAF, DC, SKOS) (Section 4.1) | ✓ Completed |
| Methodology documented (METHONTOLOGY) (Section 4.2)                                 | ✓ Completed |
| SPARQL queries for all new CQs (CQ13–CQ16) (Section 6)                              | ✓ Completed |
| V1 → V2 change summary with rationale (Section 4.3)                                 | ✓ Completed |